Console Clinic Appointment System

Java Project Brief | Difficulty: Intermediate (7/10) | Duration: 3 weeks | Individual or pairs

1. Specifying the Problem

Many small clinics still book patients by phone and write appointments in a paper diary or a shared spreadsheet. This works for a few patients a day, but as the clinic grows the same problems keep appearing:

- **Double bookings.** Two patients are given overlapping times with the same doctor, or a doctor is booked during lunch or on leave.
- **Wasted slots.** When a patient cancels, the slot usually stays empty because nobody has time to call the next person who wanted it.
- **No-shows without consequences.** Patients who repeatedly miss appointments take slots from others, and nobody tracks who they are.
- **Inconsistent charges.** Insurance patients, seniors, and walk-ins are billed differently depending on who is at the desk, and late-cancellation fees are rarely applied.
- **Poor visibility.** Doctors cannot see a clean daily schedule, and the manager cannot tell how busy each doctor is.
- **Scattered patient history.** Visit notes sit in separate files, and follow-up visits cannot be checked against the original consultation.

2. Project Idea

Build a **command-line clinic appointment system in Java**. Receptionists register patients, manage doctors and their schedules, and book, move, or cancel appointments. Patients can book and manage their own appointments. Doctors view their daily schedule and record visit notes. The system enforces every booking rule automatically, calculates fees using pricing policies, tracks no-shows, and offers cancelled slots to patients on a waiting list. All data is stored in JSON files so it survives restarts. There is no GUI and no database.

The central challenge is **time-slot logic**: working hours, breaks, leave days, durations, and overlap detection. Students also practise inheritance (users and appointment types), polymorphism (pricing), queues (waiting list), and status lifecycles.

3. How the Project Fixes the Problem

Problem	How the system solves it
Double bookings	Every booking is checked against the doctor's working hours, breaks, leave days, and existing appointments, and against the patient's own appointments. Invalid slots are rejected, and the system suggests the next free times.
Wasted slots	A waiting list holds patients who want a doctor and date range. When a slot is cancelled, the first matching patient is offered it automatically.
No-show problems	Receptionists mark no-shows. Three no-shows within 90 days blocks online booking until a receptionist lifts the block.
Inconsistent charges	Pricing strategies (Standard, Insured, Senior) calculate every fee the same way, and a late-cancellation fee is applied automatically.
Poor visibility	Doctors get a daily schedule view, and managers get reports on utilisation, no-show rate, and revenue.
Scattered history	Visit records are attached to completed appointments and are never deleted. Follow-up bookings are validated against the previous visit.

4. Project Detail

4.1 Users and authentication

- Three roles: **RECEPTIONIST**, **DOCTOR**, and **PATIENT**, all extending an abstract `User`. Login uses a user ID and password.

- Passwords are stored **hashed** (SHA-256 with a per-user salt at minimum), never in plain text.
- After 3 consecutive failed logins the account locks; only a receptionist can unlock it. A default receptionist is created on first run.
- Each role sees only its own commands. Patients can only view and change their own appointments.

4.2 Doctors and schedules

- A doctor has a name, specialty, and a **weekly schedule**: for each working day, a start time, end time, and an optional break (for example Monday to Friday, 09:00 to 17:00, break 13:00 to 14:00).
- Doctors can have **leave days** (single dates or ranges). No appointments can be booked on those days.
- The clinic uses **15-minute slots**. All appointment start times must fall on a slot boundary.
- Each doctor has a **daily capacity** (maximum appointments per day). Emergency appointments are the only type that may exceed it.

4.3 Appointment types (inheritance)

Type	Duration	Base fee	Special rules
Consultation	30 min	5,000	Standard booking rules
FollowUp	15 min	3,000	Only allowed within 14 days of a COMPLETED visit with the same doctor
Emergency	20 min	10,000	Same-day only; may use a break period; ignores daily capacity; no minimum notice

Duration, fee, and eligibility rules belong to the appointment type class, not to the booking service.

4.4 Booking rules

- The appointment must fall **inside the doctor's working hours**, outside breaks, and not on a leave day.
- It must not **overlap** another appointment for the same doctor, or another active appointment for the same patient (even with a different doctor).
- It cannot be in the past, must be at least **1 hour ahead** (except Emergency), and no more than **60 days** ahead.
- A patient may hold at most **3 active upcoming appointments**. A blocked patient cannot book until a receptionist lifts the block.
- When a slot is unavailable, the system suggests the **next 3 free slots** for that doctor.

4.5 Appointment lifecycle

Each appointment has a status, and services must allow only valid transitions:

From	To	Triggered by
BOOKED	CHECKED_IN	Receptionist checks the patient in on the day
CHECKED_IN	COMPLETED	Doctor finishes the visit and saves notes
BOOKED	CANCELLED	Patient or receptionist cancels before the start time
BOOKED	NO_SHOW	Receptionist marks it more than 15 minutes after the start time
COMPLETED, CANCELLED, NO_SHOW	(none)	Final states; any further change throws InvalidStatusTransitionException

4.6 Cancellation, rescheduling, and no-shows

- **Free cancellation** up to 24 hours before the appointment. A cancellation inside 24 hours is **late** and charges a fee of 20% of the base fee.
- **Reschedule** is one atomic operation: the new slot is validated first, and the old appointment is only cancelled if the new booking succeeds. A reschedule never leaves the patient with no appointment.
- **No-shows:** 3 no-shows within 90 days automatically block the patient from booking. The no-show history is kept even after the block is lifted.
- A cancelled slot immediately triggers the waiting list (section 4.8).

4.7 Pricing (Strategy pattern)

Patient category	Rule	Example: Consultation (5,000)
STANDARD	Patient pays the full fee	Patient pays 5,000
INSURED	Insurer covers 70%; patient pays 30%	Patient pays 1,500; insurer 3,500
SENIOR (age 65+)	20% discount on the base fee	Patient pays 4,000

Implement a `PricingPolicy` interface with one class per category. The billing service must not contain if/else chains on category. Use `BigDecimal` for all money.

4.8 Waiting list

- A patient can join a waiting list for a doctor, a date range, and an optional time window (for example mornings only).
- When a slot is cancelled, the **first matching entry** (first in, first out) is offered the slot and holds it for **2 hours**, or until 1 hour before the slot if that is sooner.
- If the patient accepts, the appointment is booked. If the offer expires or is declined, the next matching patient is offered the slot.
- Expiry is checked at startup and when each command runs (no background threads needed).

4.9 Visit records and patient history

- A doctor adds notes, a diagnosis, and prescription lines to an appointment while it is CHECKED_IN or COMPLETED.
- Records are **never deleted or overwritten**. A correction is saved as a new amendment with a timestamp and the doctor's name.
- Patients can view their own history; doctors can view the history of patients they have seen; receptionists can see appointment data but not clinical notes.

4.10 Notifications (Observer pattern)

The system fires events for: appointment booked, rescheduled, cancelled, slot offered from the waiting list, offer expired, no-show recorded, and patient blocked. Two listeners are required: a **NotificationInbox** that stores messages per user (shown at login or with `inbox`) and an **ActivityLog** that appends every event to `data/activity.log`.

4.11 Reports (receptionist)

- **Daily schedule** per doctor, with appointment times, patients, types, and statuses.
- **Utilisation:** booked minutes divided by available working minutes, per doctor and date range.
- **No-show rate** per doctor and overall, plus the list of currently blocked patients.
- **Revenue** for a date range: fees charged, insurer share, late-cancellation fees, and total collected from patients.

4.12 Command interface

Use typed commands instead of numbered menus. Quoted arguments must work, invalid input must never crash the program, and `help` lists only the commands allowed for the current role.

```
login <userId> <password>      logout      help      exit      inbox
slots <doctorId> <date> [--type=CONSULTATION]
book <doctorId> <dateTime> <CONSULTATION|FOLLOWUP|EMERGENCY>
reschedule <appointmentId> <newDateTime>      cancel <appointmentId>
myappointments      waitlist join <doctorId> <fromDate> <toDate> [--time=MORNING]
waitlist accept <offerId>      waitlist leave <entryId>
patient add <name> <dob> <STANDARD|INSURED> [--insurer=..]     patient unblock <id>
doctor add <name> <specialty>      doctor schedule <id> <day> <start> <end> [--break=13:00-14:00]
doctor leave <id> <fromDate> [<toDate>]      checkin <id>      noshow <id>
visit note <appointmentId> "<text>"      history <patientId>
report daily <doctorId> <date> | utilisation | noshow | revenue [--from=..] [--to=..]
```

Sample output of the `slots` command:

```
Dr. Bello (General Practice) - Monday 2026-11-02
09:00  09:15  09:30  [10:00 BOOKED]  10:30  10:45  11:00  ...  [13:00-14:00 BREAK]  14:00  ...
Free consultation slots: 14 of 28     Daily capacity used: 6 of 20
```

4.13 A worked example

Step	What happens
1	An INSURED patient books a Consultation with Dr. Bello for Monday 2026-11-02 at 10:00. Fee 5,000: the patient pays 1,500 and the insurer 3,500. Status: BOOKED.
2	A second patient tries 10:15 with the same doctor. The system rejects it as an overlap and suggests 10:30, 10:45, and 11:00.
3	The second patient joins the waiting list for Dr. Bello on Monday morning, then books 10:30.
4	On Sunday evening the first patient cancels. This is less than 24 hours before, so a late fee of 20% of 5,000 = 1,000 is charged. The slot is offered to the waiting-list patient for 2 hours.
5	The offer is accepted, and a new appointment is booked at 10:00. Both patients receive inbox notifications.
6	On Monday the doctor completes the visit, adds notes, and the status becomes COMPLETED. A FollowUp can now be booked within 14 days.

4.14 Data storage (JSON) and thread safety

- Use **Gson**. Store files in a `data/` folder: users, doctors, appointments, waiting list, visit records, and the activity log.
- Dates and times are ISO-8601 strings such as "2026-11-02T10:00:00". Money is `BigDecimal` stored as strings.
- Users, appointment types, and pricing categories store a `type` field. Factories rebuild the correct subclass when loading.
- Load everything at startup and save after every state-changing operation. Write to a temporary file first, then move it over the real file so a crash cannot corrupt data.
- IDs continue from the last saved value after a restart. A missing file is created; a corrupt file shows a clear message and exits cleanly.
- Two users may try to book the same slot at the same moment. Booking, cancelling, and rescheduling must be thread-safe (`synchronized` or `ReentrantLock`) so exactly one succeeds.

5. Project Requirements

5.1 Technical requirements

- Java 17 or newer, built with Maven. Gson is the only third-party library allowed.
- No GUI and no database. All interaction is through the console; all storage is JSON files.
- Use `java.time` (`LocalDateTime`, `LocalTime`, `Duration`, `DayOfWeek`). Never call `LocalDateTime.now()` inside a service; use an injected `Clock`.
- Never use `double` or `float` for money.

5.2 Design requirements

- **Inheritance and polymorphism:** abstract `User` (Receptionist, Doctor, Patient) and abstract `AppointmentType` (Consultation, FollowUp, Emergency).
- **Patterns:** Strategy (pricing), Factory (users, appointment types, policies), Observer (notifications), Repository (data access). The Command pattern for CLI commands is encouraged.
- **Layering:** the CLI talks to services only; services depend on repository *interfaces*, not JSON classes.
- **Custom exceptions** (at minimum): DoctorNotFound, PatientNotFound, AppointmentNotFound, SlotUnavailable, OutsideWorkingHours, DoctorOnLeave, PatientBlocked, BookingLimitExceeded, FollowUpNotEligible, InvalidStatusTransition, Authentication, UnauthorizedAction. They are caught in one place (the CLI layer) and shown as friendly messages.

5.3 Required package structure

```
com.clinic
  model/       User, Receptionist, Doctor, Patient, Appointment, AppointmentType,
               WorkingDay, LeavePeriod, WaitlistEntry, VisitRecord, Role
  service/     AuthService, SchedulingService, BookingService, WaitlistService,
               BillingService, VisitService, ReportService
  scheduling/  TimeRange, SlotFinder, ScheduleRules
  pricing/     PricingPolicy + implementations
  event/       ClinicEvent, EventBus, listeners
  repository/  repository interfaces + JSON implementations
  factory/     UserFactory, AppointmentTypeFactory, PricingPolicyFactory
  cli/         CommandParser, Command classes, ConsoleUI
  exception/   custom exceptions        util/  PasswordHasher, Validators, IdGenerator
```

6. Project Deliverables

- **Source code** in a Git repository with regular, meaningful commits.
- **README.md** explaining how to build and run the project, the full command list, and the main design decisions.
- **UML class diagram** (hand-drawn or tool-made) showing the user and appointment type hierarchies, pricing policies, and services, plus a **status diagram** of the appointment lifecycle.
- **Seed data** loaded on first run: at least 4 doctors with different schedules, 10 patients (including insured and senior), and 15 appointments in various states.
- **Demo** (5 minutes, live or recorded) showing an overlap rejection, a late cancellation with a fee, a waiting-list offer, a no-show block, and a restart with data intact.

6.1 Milestones

Week	Deliverable
1	Models, exceptions, schedule rules and slot finder, appointment types, pricing policies, booking service using in-memory storage
2	Cancellation, rescheduling, no-show rules, waiting list, events and notifications, JSON storage, authentication, command parser, and CLI
3	Visit records, reports, thread safety, polish, documentation, and demo

6.2 Evaluation (100 points)

Area	Points	Area	Points
Correct scheduling, booking, cancellation, and waiting-list rules	30	Error handling and CLI usability	10
OOP design and patterns	25	Code quality and naming	5
Data storage and integrity (JSON)	15	Documentation, diagrams, Git history	5
Date and time handling (Clock, slots, durations)	10		

Bonus ideas: recurring appointments (every week for 4 weeks), multiple doctors per specialty with automatic assignment, a calendar-style weekly view in the console, patient reminders at login for appointments within 24 hours, and an export of a patient's history to a JSON file.

Tip: build a small `TimeRange` class with an `overlaps()` method first and get it completely right. Almost every rule in this project (breaks, leave, double booking) depends on it. Discuss ideas freely with classmates, but write your own code.